树与二叉树全面笔记

目录

1. 树的基本概念
2. 二叉树
3. 二叉树的存储结构
4. 二叉树的遍历
5. 遍历的非递归实现
6. 由遍历序列恢复二叉树
7. 二叉树的基本操作
8. 二叉查找树（BST）
9. 平衡二叉树（AVL）
10. 树与二叉树的转换
11. 树的遍历
12. 线索二叉树
13. 哈夫曼树与哈夫曼编码
14. 词频统计问题对比
15. 常考题型与易错点

一、树的基本概念

1.1 为什么需要树

线性结构（数组、链表、栈、队列）是一对一的关系，而现实中有大量**一对多**的关系需要处理：

线性结构	树结构
第一个元素（无前驱）	**根结点**（无前驱）
最后一个元素（无后继）	**叶子结点**（无后继）
其他元素（一个前驱、一个后继）	其他结点（**一个前驱、多个后继**）

生活类比： 文件系统目录、公司组织架构、家族族谱、网页 DOM 结构。

词频统计问题暴露了线性结构的局限：

- 顺序表：查找快（折半）但插入慢（移动大量数据）
- 链表：插入快但查找慢（每次从头查）
- **树结构：查找和插入都能做到 $O(\log_2 n)$**，是最优选择

1.2 树的定义

树是由 **n ≥ 0** 个结点组成的有穷集合 D，以及结点之间关系组成的集合构成的结构，记为 T。

- **n = 0** → 空树

- **非空树**：有一个特殊的根结点 root，其余结点被分成 **m ≥ 0** 个不相交的子集 $D_1, D_2, ..., D_m$，每个子集分别构成一棵树，称为根结点的**子树**

> 树的定义是**递归的**——树的子树还是树。

1.3 树的特点（逻辑上）

1. **有且仅有一个根结点**（无前驱）
2. 除根结点外，**每个结点有且仅有一个直接前驱**
3. 每个结点可以**有多个后继**（包括根结点）

```
        ①          ← 根（无前驱）
      / | \
    ②  ③  ④       ← 每个结点一个前驱
   / \    / \ \
  ⑤  ⑥  ⑦ ⑧ ⑨    ← 叶子（无后继）
```

1.4 树的表示方法

方法	示例	特点
树形图	树状分支图	最直观
文氏图	嵌套圆圈	显示包含关系
凹入表示法	缩进层次	文件目录
嵌套括号（广义表）	`A(B(E,F,G),C(H),D(I,J))`	紧凑，适合输入

> 嵌套括号法写法：**根(子树1, 子树2, ...)**，子树同样递归。

1.5 树的基本术语

术语	定义	示例说明
结点的度	该结点拥有的子树数目	上图中 ① 的度为 3
树的度	所有结点度的最大值	上图中树的度为 3
叶子（叶结点）	度为 0 的结点	⑤ ⑥ ⑦ ⑧ ⑨
分支结点	度非 0 的结点	① ② ③ ④
层次	根为第 1 层，孩子为第 i+1 层	⑤ 是第 3 层
深度（高度）	结点所处的最大层次数	上图深度为 3
路径	结点序列 $d_1, d_2, ..., d_k$，d_i 是 d_{i+1} 的双亲	①→②→⑤ 是一条路径
祖先/子孙	路径上的结点互为祖先/子孙	① 是 ⑤ 的祖先
有序树/无序树	子树有无左右顺序之分	二叉树是**有序树**

1.6 树的度和二叉树的关系

> **树的度 ≠ 二叉树的度。** 度为 2 的树不一定是二叉树，因为二叉树要求子树有严格的左右之分。二叉树是**有序树**。

1.7 几个关键结论

- 从根结点到树中任意其他结点均有**唯一路径**
- 树中**不存在回路**
- n 个结点的树有 **n-1 条边**（分支）

二、二叉树

2.1 二叉树的定义

二叉树是 **n ≥ 0** 个结点的有穷集合 D 与关系的集合 R 构成的结构，记为 T。

- **n = 0** → 空二叉树
- **非空二叉树**：包含一个**根结点**和两棵**不相交**的、分别称为**左子树**与**右子树**的二叉树

> 二叉树的定义也是**递归的**。

2.2 二叉树 vs 树

对比	树	二叉树
子树个数	任意（0 ~ ∞）	最多 2 个（左、右）
子树顺序	可无序	**严格区分左右**
度	任意	≤ 2
空树	允许	允许

2.3 二叉树的 5 种基本形态

```
1）空二叉树          2）只有根          3）根+左子树
      ∅                 ●                  ●
                                          /
                                         ●

4）根+右子树         5）根+左子树+右子树
      ●                 ●
       \               / \
        ●             ●   ●
```

> **易错：** 三个结点的二叉树有 **5 种**形态，而三个结点的树只有 **1 种**形态（因为树不区分左右）。

```
三个结点的二叉树(5种):
  1)   2)     3)     4)     5)
  ●    ●      ●      ●      ●
 /      \    / \    /        \
●        ●  ●   ●  ●          ●
/          \       /            \
●           ●     ●             ●
```

2.4 两种特殊二叉树

满二叉树

所有叶子结点都在最底层，非叶子结点都有两棵子树。

```
          ①
        /   \
      ②      ③
     / \    / \
    ④  ⑤ ⑥  ⑦
```

- 深度为 h 的满二叉树有 **2^h - 1** 个结点

完全二叉树

- 最下面两层结点的度可以小于 2
- 最下层叶子结点**从左至右依次排列**
- 满二叉树是特殊的完全二叉树

```
完全二叉树 ✓              非完全二叉树 ✗
     ①                         ①
    / \                       / \
   ②   ③                     ②   ③
  / \                       /     \
 ④   ⑤                     ④       ⑤
```

判断技巧： 按层编号后，结点连续无空缺就是完全二叉树。

2.5 二叉树的 6 个性质（重点 ⭐⭐⭐⭐⭐）

性质 1

> **具有 n 个结点的非空二叉树，共有 n-1 个分支。**

- 每个结点（根除外）都有一个双亲，每个双亲关系对应一个分支。

性质 2

> **非空二叉树的第 i 层（i ≥ 1）最多有 2^{i-1} 个结点。**

层数	最多结点数
1	$2^0 = 1$
2	$2^1 = 2$
3	$2^2 = 4$
4	$2^3 = 8$
…	…

性质 3

> **深度为 h 的非空二叉树最多有 $2^h - 1$ 个结点。**

即等比数列求和：$1 + 2 + 4 + \ldots + 2^{h-1} = 2^h - 1$

性质 4 ⭐（最重要）

> **若非空二叉树有 n_0 个叶子结点，n_2 个度为 2 的结点，则 $n_0 = n_2 + 1$。**

证明：

- 设总结点数 = n，度 1 结点数 = n_1
- $n = n_0 + n_1 + n_2$ ①
- 从分支角度看：总分支数 = n - 1（性质 1）
- 总分支数 = $2n_2 + n_1$ ②（度为 2 的贡献 2 条，度为 1 的贡献 1 条）
- 由 ① ② 得：$n_0 + n_1 + n_2 - 1 = 2n_2 + n_1$
- **$n_0 = n_2 + 1$** ✅

> **快速记忆：叶子比 2 度结点多 1 个。**

性质 5

> **具有 n 个结点的非空完全二叉树的深度为 $h = \lfloor \log_2 n \rfloor + 1$。**

推导：

- 深度 h-1 的满二叉树结点数：$2^{h-1} - 1$
- 深度 h 的满二叉树结点数：$2^h - 1$
- 完全二叉树结点数 n 满足：$2^{h-1} - 1 < n \le 2^h - 1$
- $h = \lfloor \log_2 n \rfloor + 1$

性质 6 ⭐（数组存储用）

完全二叉树按层次从左到右编号（从 1 开始），对编号为 i 的结点：

1. **i = 1** → 根结点
2. **i > 1** → 双亲编号为 **⌊i / 2⌋**
3. **2i > n** → 无左孩子；否则左孩子编号为 **2i**

4. **2i + 1 > n** → 无右孩子；否则右孩子编号为 **2i + 1**

```
              ①
            /   \
         ②      ③
        / \    / \
       ④  ⑤ ⑥  ⑦       ← 编号连续，完全二叉树

关系：
  ②的双亲 = ⌊2/2⌋ = 1 ✓
  ②的左孩子 = 2×2 = 4 ✓
  ②的右孩子 = 2×2+1 = 5 ✓
  ④的左孩子 = 2×4 = 8 > 7 → 无左孩子 ✓
```

> 这个性质是**完全二叉树能用数组高效存储**的理论基础。

2.6 性质练习题详解

练习 1： 一棵二叉树有 1001 个结点，且无度为 1 的结点，则叶子结点个数为？

解：

- $n = 1001$，$n_1 = 0$
- $n_0 + n_2 = 1001$（因为 $n_1 = 0$）
- $n_0 = n_2 + 1$（性质 4）
- $\Rightarrow n_0 + (n_0 - 1) = 1001 \Rightarrow 2n_0 = 1002 \Rightarrow$ **$n_0 = 501$**
- 答案：**D) 501**

练习 2： 深度为 6 的完全二叉树，第 6 层有 3 个叶子结点，则该二叉树共有叶子结点数为？

解：

- 深度 6，前 5 层是满的
- 前 5 层叶子数 = 第 5 层结点数 = $2^4 = 16$（因为第 6 层只有 3 个，第 5 层结点不全是叶子）
- 实际上前 5 层的叶子数：完全二叉树第 6 层有 3 个结点，它们占用了第 5 层的 2 个结点作为双亲（也可能是 1 个或 2 个）
- 第 6 层叶结点 = 3
- 总叶结点 = 第 5 层叶结点 + 第 6 层叶结点
- 第 5 层有 $2^4 = 16$ 个结点，其中第 6 层的 3 个结点对应第 5 层的 2 个双亲
- 第 5 层叶结点 = 16 - 2 = 14
- 总叶结点 = 14 + 3 = **17**
- 答案：**A) 17**

等等，这个解题需要重新思考。完全二叉树中，第 6 层有 3 个结点，所以第 5 层中只有 2 个结点有孩子（因为 3 个孩子至少需要 2 个父亲），所以第 5 层的叶子结点数为 16 - 2 = 14。总叶结点数 = 14 + 3 = 17。✓

练习 3： 具有 767 个结点的完全二叉树，其叶子结点数为？

解：

- n = 767
- 由性质 5：h = $\lfloor \log_2 767 \rfloor + 1 = \lfloor 9.58 \rfloor + 1 = 9 + 1 = 10$
- 前 9 层满：$2^9 - 1 = 511$ 个结点
- 第 10 层结点数 = 767 - 511 = 256
- 这些叶子都是第 10 层的叶子
- 但第 9 层中，有 [256/2] = 128 个结点有孩子（不是叶子）
- 第 9 层总结点数 = $2^8 = 256$
- 第 9 层叶子数 = 256 - 128 = 128
- 总叶子数 = 128 + 256 = **384**
- 答案：**B) 384**

三、二叉树的存储结构

3.1 顺序存储（数组）

完全二叉树的顺序存储

利用性质 6，按层编号存入数组，编号 i → 下标 i-1。

```
完全二叉树：          数组存储：
     ①              index: 0  1  2  3  4  5  6
    / \              value: ① ② ③ ④ ⑤ ⑥ ⑦
  ②   ③
 / \  / \
④  ⑤⑥  ⑦
```

优点： 紧凑，无空间浪费，可通过下标直接计算父子关系

一般二叉树的顺序存储

补全为完全二叉树再存储，空缺位置留空。

```
一般二叉树：          补全为完全二叉树：
     ①                     ①
    / \                   / \
  ②   ③                 ②   ③
    /                     \  /
  ④                      ④ □ □
                        /
                       □
```

缺点： 可能造成**巨大的空间浪费**（极端情况：深度为 h 的右斜树只有 h 个结点，却需要 $2^h - 1$ 个数组空间）

3.2 链式存储

二叉链表（最常用）

```c
typedef struct BTNode {
    datatype data;
    struct BTNode *lchild;   // 左孩子指针
    struct BTNode *rchild;   // 右孩子指针
} BTNode, *BTNodePtr;
```

```
结点结构：
┌──────┬──────┬──────┐
│lchild│ data │rchild│
└──────┴──────┴──────┘
   ↓             ↓
左子树         右子树
```

n 个结点的二叉树，有 n+1 个空指针域。

- 总指针域 = 2n
- 非空指针域 = n - 1（分支数）
- 空指针域 = 2n - (n - 1) = **n + 1**

三叉链表

```c
typedef struct BTNode {
    datatype data;
    struct BTNode *parent;   // 双亲指针
    struct BTNode *lchild;   // 左孩子
    struct BTNode *rchild;   // 右孩子
} BTNode;
```

优点： 可以方便地找到双亲结点（二叉链表找双亲需要遍历）

三种存储结构对比

结构	空间	找孩子	找双亲	适用
顺序存储（完全二叉树）	紧凑	公式 O(1)	公式 O(1)	完全/满二叉树
顺序存储（一般二叉树）	可能极浪费	公式 O(1)	公式 O(1)	不推荐
二叉链表	2n 指针	lchild/rchild O(1)	遍历 O(n)	**最常用**
三叉链表	3n 指针	lchild/rchild O(1)	parent O(1)	需要频繁找双亲

四、二叉树的遍历

4.1 遍历的概念

遍历： 按照一定次序**访问每个结点一次且仅一次**。

4.2 前序遍历（DLR）

顺序：根 → 左子树 → 右子树

```
void preorder(BTNodePtr t) {
    if (t != NULL) {
        VISIT(t);                   // 访问根
        preorder(t->lchild);        // 遍历左子树
        preorder(t->rchild);        // 遍历右子树
    }
}
```

```
遍历结果：① ② ④ ⑤ ③ ⑥
        ①
       / \
     ②   ③
    / \  /
   ④  ⑤⑥
```

4.3 中序遍历（LDR）

顺序：左子树 → 根 → 右子树

```
void inorder(BTNodePtr t) {
    if (t != NULL) {
        inorder(t->lchild);        // 遍历左子树
        VISIT(t);                   // 访问根
        inorder(t->rchild);        // 遍历右子树
    }
}
```

```
遍历结果：④ ② ⑤ ① ⑥ ③
        ①
       / \
     ②   ③
    / \  /
   ④  ⑤⑥
```

4.4 后序遍历（LRD）

顺序：左子树 → 右子树 → 根

```c
void postorder(BTNodePtr t) {
    if (t != NULL) {
        postorder(t->lchild);    // 遍历左子树
        postorder(t->rchild);    // 遍历右子树
        VISIT(t);                 // 访问根
    }
}
```

```
遍历结果：④ ⑤ ② ⑥ ③ ①
        ①
      / \
    ②   ③
   / \  /
  ④  ⑤⑥
```

4.5 层次遍历（广度优先）

顺序：从上到下、从左到右，用队列实现

```c
#define MAX 100

void layerorder(BTNodePtr t) {
    BTNodePtr queue[MAX], p;
    int front = 0, rear = 0;

    if (t != NULL) {
        queue[rear++] = t;           // 根入队
        while (front < rear) {
            p = queue[front++];      // 出队
            VISIT(p);                // 访问
            if (p->lchild != NULL)
                queue[rear++] = p->lchild;  // 左孩子入队
            if (p->rchild != NULL)
                queue[rear++] = p->rchild;  // 右孩子入队
        }
    }
}
```

```
遍历结果：① ② ③ ④ ⑤ ⑥
        ①
      / \
    ②   ③
   / \  /
  ④  ⑤⑥
```

4.6 四种遍历对比

遍历方式	顺序	实现工具	特点
前序	根→左→右	栈（递归/非递归）	第一个访问的是根
中序	左→根→右	栈（递归/非递归）	BST 中序为升序
后序	左→右→根	栈（递归/非递归）	最后一个访问的是根
层次	逐层从左到右	队列	广度优先

4.7 表达式树遍历示例

表达式 `(a + b * (c - d)) - e / f` 的二叉树：

```
        -
      /   \
     +     /
    / \   / \
   a   * e   f
      / \
     b   -
        / \
       c   d
```

遍历方式	结果
前序	`- + a * b - c d / e f`（**前缀表达式**）
中序	`a + b * c - d - e / f`（**中缀表达式**，缺括号）
后序	`a b c d - * + e f / -`（**后缀表达式/RPN**）
层次	`- + / a * e f b - c d`

> 前序遍历结果就是**前缀表达式**（波兰式），后序遍历结果就是**后缀表达式**（逆波兰式）。

五、遍历的非递归实现

5.1 中序非递归遍历

核心思想： 用栈模拟递归过程——沿着左子树一路入栈，到底后出栈访问，再转向右子树。

```
#define M 1024

void inorder_nonrecursive(BTNodePtr t) {
    BTNodePtr stack[M], p = t;
    int top = -1;
```

```
    if (t != NULL) {
        do {
            while (p != NULL) {        // 一直往左走
                stack[++top] = p;      // 沿途结点入栈
                p = p->lchild;
            }
            p = stack[top--];          // 出栈
            VISIT(p);                  // 访问
            p = p->rchild;             // 转向右子树
        } while (!(p == NULL && top == -1));
        // 等价于：while (p != NULL || top != -1)
    }
}
```

手动模拟：

```
树：
     ①
      \
       ②
      /
    ③

操作                栈          输出
p=①, p≠NULL → push ①  [①]
p=①→lchild=NULL
pop ① → VISIT(①)      []          ①
p=①→rchild=②
p=②≠NULL → push ②    [②]
p=②→lchild=③ ≠NULL → push ③  [②,③]
p=③→lchild=NULL
pop ③ → VISIT(③)      [②]       ① ③
p=③→rchild=NULL
pop ② → VISIT(②)      []        ① ③ ②
p=②→rchild=NULL
p==NULL && top==-1 → 结束
```

5.2 前序非递归遍历

```
void preorder_nonrecursive(BTNodePtr t) {
    BTNodePtr stack[M], p = t;
    int top = -1;

    if (t != NULL) {
        stack[++top] = t;
        while (top >= 0) {
            p = stack[top--];
            VISIT(p);                      // 先访问根
            if (p->rchild != NULL)         // 注意：先右后左（栈是 LIFO）
```

```
                stack[++top] = p->rchild;
            if (p->lchild != NULL)
                stack[++top] = p->lchild;
        }
    }
}
```

5.3 三种遍历的非递归统一写法 ⭐

```
// 前序、中序、后序的非递归统一框架
// 区别仅在于 VISIT 的位置不同
void traversal(BTNodePtr t) {
    BTNodePtr stack[M], p = t;
    int top = -1;

    while (p != NULL || top != -1) {
        while (p != NULL) {
            // 前序 VISIT 写在这里
            stack[++top] = p;
            p = p->lchild;
        }
        p = stack[top--];
        // 中序 VISIT 写在这里
        p = p->rchild;
        // 后序较复杂，需要标记访问过右子树
    }
}
```

> 后序非递归最复杂，需要记录每个结点右子树是否已访问过（可用一个标记位或额外栈）。

六、由遍历序列恢复二叉树

6.1 核心规律

已知序列	能否唯一恢复	方法
前序 + 中序	✅ **能唯一确定**	前序定根，中序分左右
后序 + 中序	✅ **能唯一确定**	后序定根，中序分左右
前序 + 后序	❌ **不能唯一确定**	无法区分左右子树

6.2 前序 + 中序 → 恢复二叉树

示例：

```
前序：A B D E C G H I F
```

```
中序：D E B A H G I C F
```

恢复步骤：

```
第1步：前序第一个 = A → 根结点
      中序 A 左边 = D E B → 左子树
      中序 A 右边 = H G I C F → 右子树

第2步：左子树部分
      前序中左子树顺序：B D E → B 是左子树的根
      中序 B 左边 = D E → B 的左子树
      中序 B 右边 = （空）→ B 无右子树
        D E 中：前序 D E → D 是根
        中序 D 右边 = E → D 的右子树

第3步：右子树部分
      前序中右子树顺序：C G H I F → C 是右子树的根
      中序 C 左边 = H G I → C 的左子树
      中序 C 右边 = F → C 的右子树
        H G I 中：前序 G H I → G 是根
        中序 G 左边 = H → G 的左子树
        中序 G 右边 = I → G 的右子树

最终二叉树：
        A
       / \
      B   C
     /   / \
    D   G   F
     \ / \
      E H I
```

验证后序： 对恢复的树做后序遍历 → E D B H I G F C A ✓

6.3 后序 + 中序 → 恢复二叉树

方法与上类似，但**后序最后一个元素是根**。

七、二叉树的基本操作

7.1 求二叉树的高度

```
int max(int x, int y) {
    return (x > y) ? x : y;
}

int height(BTNodePtr t) {
    if (t == NULL)
```

```
        return 0;
    return 1 + max(height(t->lchild), height(t->rchild));
}
```

> ⚠️ **原代码错误修正：** 原课件写的是 `int max(x,y) { if((x >y) return x; else return y; }`，缺少参数类型和括号。正确写法如上。

7.2 求结点数

```
int countNodes(BTNodePtr t) {
    if (t == NULL)
        return 0;
    return 1 + countNodes(t->lchild) + countNodes(t->rchild);
}
```

7.3 求叶子结点数

```
int countLeaves(BTNodePtr t) {
    if (t == NULL)
        return 0;
    if (t->lchild == NULL && t->rchild == NULL)
        return 1;
    return countLeaves(t->lchild) + countLeaves(t->rchild);
}
```

7.4 拷贝二叉树（前序思路）

```
BTNodePtr copyTree(BTNodePtr src) {
    BTNodePtr obj;
    if (src == NULL) {
        return NULL;
    }
    obj = (BTNodePtr)malloc(sizeof(BTNode));
    obj->data = src->data;                  // 拷贝根
    obj->lchild = copyTree(src->lchild);    // 拷贝左子树
    obj->rchild = copyTree(src->rchild);    // 拷贝右子树
    return obj;
}
```

7.5 删除二叉树（后序思路）

```
void destroyTree(BTNodePtr p) {
    if (p != NULL) {
        destroyTree(p->lchild);    // 先删左子树
```

```
        destroyTree(p->rchild);    // 再删右子树
        free(p);                   // 最后删根
        // p = NULL;  // 注意：这里 p 是形参，置 NULL 不影响实参，
                      // 需在调用处手动置 NULL
    }
}
```

> **为什么用后序？** 必须先删孩子再删双亲，否则先删双亲就找不到孩子了。

7.6 输出叶结点及高度

题目： 读入整数，按左<根≤右构建 BST，输出每个叶结点的值和高度（根高度为 1）。

```
void printLeafNode(BTNodePtr root, int height) {
    if (root != NULL) {
        if (root->left == NULL && root->right == NULL) {
            printf("%d %d\n", root->data, height);  // 叶结点，输出
        }
        printLeafNode(root->left, height + 1);   // 遍历左子树
        printLeafNode(root->right, height + 1);  // 遍历右子树
    }
}
```

样例输入输出：

```
输入：13
50 38 30 64 58 40 10 73 70 50 60 100 35

输出：
10 4
35 4
40 3
50 4
60 4
70 4
100 4
```

八、二叉查找树（BST）

8.1 定义

二叉查找树（Binary Search Tree, BST） 要么是空树，要么满足：

1. **左子树**所有结点值 < 根值
2. **右子树**所有结点值 ≥ 根值（有的定义用 >，取决于实现）
3. 左、右子树本身也是 BST

> BST 的中序遍历结果是**递增序列**（重要性质 ⭐）。

8.2 BST 的查找

非递归版（效率更高）：

```
BTNodePtr searchBST(BTNodePtr t, datatype key) {
    BTNodePtr p = t;
    while (p != NULL) {
        if (key == p->data)
            return p;               // 找到
        if (key > p->data)
            p = p->rchild;          // 去右子树找
        else
            p = p->lchild;          // 去左子树找
    }
    return NULL;                    // 没找到
}
```

递归版：

```
BTNodePtr searchBST_recursive(BTNodePtr t, datatype key) {
    if (t == NULL)
        return NULL;
    if (key == t->data)
        return t;
    if (key > t->data)
        return searchBST_recursive(t->rchild, key);
    else
        return searchBST_recursive(t->lchild, key);
}
```

> **时间复杂度：** 平均 $O(\log_2 n)$，最坏 $O(n)$（退化为单链表时）。

8.3 BST 的插入（建立）

```
BTNodePtr insertBST(BTNodePtr p, datatype item) {
    if (p == NULL) {
        p = (BTNodePtr)malloc(sizeof(BTNode));
        p->data = item;
        p->lchild = p->rchild = NULL;
    } else if (item < p->data) {
        p->lchild = insertBST(p->lchild, item);    // 插到左子树
    } else if (item > p->data) {
        p->rchild = insertBST(p->rchild, item);    // 插到右子树
    }
    // item == p->data → 视需求处理（不插/计数）
```

```
    return p;
}
```

建立 BST（逐点插入）：

```
BTNodePtr root = NULL;
int data[] = {50, 38, 30, 64, 58, 40, 10, 73, 70, 50, 60, 100, 35};
int n = 13;

for (int i = 0; i < n; i++) {
    root = insertBST(root, data[i]);
}
```

BST 建立过程：

```
插入 50:  50
插入 38:  50
         /
       38
插入 30:  50
         /
       38
       /
     30
插入 64:    50
          /  \
        38   64
       /
     30
... 最终:
           50
         /    \
       38      64
      /  \    /  \
    30   40  58  73
   /  \    \   \   \
  10  35   50  60  100
```

8.4 BST 的删除 ⭐⭐⭐

情况	操作	图示
① 叶结点	直接删除	删除 30：50→38→NULL
② 只有左子树	左子树的根取代	删除 64：50→右=58
③ 只有右子树	右子树的根取代	—
④ 左右子树都有	用**右子树最小值**或**左子树最大值**取代	删除 38：用 40（右子树最小）取代

```
BTNodePtr deleteBST(BTNodePtr root, datatype key) {
    if (root == NULL) return NULL;

    if (key < root->data)
        root->lchild = deleteBST(root->lchild, key);
    else if (key > root->data)
        root->rchild = deleteBST(root->rchild, key);
    else {  // 找到要删除的结点
        // 情况① ② ③：最多一个孩子
        if (root->lchild == NULL) {
            BTNodePtr temp = root->rchild;
            free(root);
            return temp;
        }
        if (root->rchild == NULL) {
            BTNodePtr temp = root->lchild;
            free(root);
            return temp;
        }
        // 情况④：左右子树都有
        // 找右子树的最小值
        BTNodePtr temp = root->rchild;
        while (temp->lchild != NULL)
            temp = temp->lchild;
        root->data = temp->data;                              // 用最小值取代
        root->rchild = deleteBST(root->rchild, temp->data);   // 删除那个最小值结点
    }
    return root;
}
```

删除 38（左右都有）示例：

```
 删除前:          删除后:
    50               50
   /  \             /  \
 38   64          40   64
 / \  / \        /    / \
30 40 58 73    30    58 73
   \            \
   50           50
```

8.5 查找祖先结点

给定一个值，输出从根到该结点的路径上所有结点的值：

```
void printAncestors(BTNodePtr t, datatype item) {
    BTNodePtr p = t;
    while (p != NULL && p->data != item) {
```

```
        printf("%d ", p->data);     // 输出当前结点（不是目标）
        if (item > p->data)
            p = p->rchild;
        else
            p = p->lchild;
    }
    // 找到或没找到都会停
}
```

8.6 BST 应用：词频统计

对比线性结构的劣势：

方案	查找	插入	空间
顺序表（有序数组）	折半 $O(\log_2 n)$	O(n) 移动数据	固定，可能溢出或浪费
链表	O(n) 从头遍历	O(1) 插入	动态
BST	**$O(\log_2 n)$**	**$O(\log_2 n)$**	**动态**

BST 做词频统计的结点定义和操作：

```
struct tnode {
    char word[32];            // 单词
    int count;                // 出现次数
    struct tnode *left;       // 左子树（字典序小）
    struct tnode *right;      // 右子树（字典序大）
};

// 插入或更新单词
struct tnode *addWord(struct tnode *p, char *w) {
    int cmp;
    if (p == NULL) {
        p = (struct tnode *)malloc(sizeof(struct tnode));
        strcpy(p->word, w);
        p->count = 1;
        p->left = p->right = NULL;
    } else if ((cmp = strcmp(w, p->word)) == 0) {
        p->count++;                     // 已有单词，次数+1
    } else if (cmp < 0) {
        p->left = addWord(p->left, w);  // 插入左子树
    } else {
        p->right = addWord(p->right, w);// 插入右子树
    }
    return p;
}

// 按字典序输出（中序遍历就是升序！）
void printWords(struct tnode *p) {
    if (p != NULL) {
        printWords(p->left);            // 先输出左（小）
```

```
        printf("%s  %d\n", p->word, p->count);
        printWords(p->right);          // 再输出右（大）
    }
}
```

> BST 的中序遍历结果就是**有序（字典序）输出**！

九、平衡二叉树（AVL）

9.1 为什么需要 AVL

BST 在极端情况下会退化为**单链表**（如插入 1,2,3,4,5），查找效率降到 O(n)。

9.2 平衡因子

平衡因子 = 左子树高度 - 右子树高度

AVL 要求每个结点的平衡因子只能是 **-1、0、1**。

```
平衡结点:          不平衡结点:
    50               50(BF=2)
   /  \             /
 38    64         38
(BF=0)          (BF=1)
               /
             30
            (BF=0)
```

9.3 四种失衡类型及调整

类型	情形	调整方式
LL	左子树的左子树插入	**右旋转**
RR	右子树的右子树插入	**左旋转**
LR	左子树的右子树插入	**先左旋再右旋**
RL	右子树的左子树插入	**先右旋再左旋**

```
LL 右旋转示例:
      50                38
     /                 / \
    38      →        30  50
   /
  30
```

> AVL 确保了树的高度始终为 $O(\log_2 n)$，查找/插入/删除都是 $O(\log_2 n)$。

十、树与二叉树的转换

10.1 树 → 二叉树

三步法（左孩子右兄弟）：

```
原始树：      ①            ①            ①                  ①
            /|\          / \          / \                / \
          ② ③ ④   →    ② ③ ④   →   ②   ③       →      ②   ③
          / \          /                 \                \    \
         ⑤   ⑥       ⑤                   ④                ⑤   ④
                                                  / \              \
                                                 ⑤   ⑥              ⑥
```

1. **加线：** 相邻兄弟之间加一条连线
2. **删线：** 只保留每个结点与最左孩子的连线，删除与其他孩子的连线
3. **旋转：** 顺时针旋转 45°，根无右子树

10.2 树林 → 二叉树

1. 每棵树分别转成二叉树
2. 后一棵树的根作为前一棵树的**右孩子**

10.3 二叉树 → 树（逆过程）

1. 左孩子的右链全部与双亲连线
2. 删除双亲与右孩子的连线
3. 规整图形

十一、树的遍历

11.1 前序遍历

先访问根，再依次遍历各棵子树（深度优先）。

```c
#define MAXD 20  // 树的最大度数

void preorderTree(TNodePtr t) {
    int i;
    if (t != NULL) {
        VISIT(t);
        for (i = 0; i < MAXD; i++) {
            if (t->next[i] != NULL)
                preorderTree(t->next[i]);
        }
```

```
    }
}
```

11.2 后序遍历

先依次遍历各棵子树，再访问根。

11.3 层次遍历（广度优先，BFS）

```
void levelOrderTree(TNodePtr t) {
    TNodePtr p;
    int i;
    if (t != NULL) {
        enQueue(t);
        while (!isEmpty()) {
            p = deQueue();
            VISIT(p);
            for (i = 0; i < MAXD; i++) {
                if (p->next[i] != NULL)
                    enQueue(p->next[i]);
            }
        }
    }
}
```

> **注意：** 一般的树**没有中序遍历**（因为子树不止两个，无所谓"中"）。

十二、线索二叉树

12.1 为什么需要线索

n 个结点的二叉链表有 **n+1 个空指针域**，这些空间被浪费了。

线索二叉树利用这些空指针：

- 左空指针 → 指向**前驱**（遍历顺序中的前一个）
- 右空指针 → 指向**后继**（遍历顺序中的下一个）

12.2 结点结构

```
┌──────┬──────┬──────┬──────┬──────┐
│ ltag │ lchild │ data │ rchild │ rtag │
└──────┴──────┴──────┴──────┴──────┘

ltag = 0 → lchild 指向左孩子
ltag = 1 → lchild 指向前驱（线索）
```

```
rtag = 0 → rchild 指向右孩子
rtag = 1 → rchild 指向后继（线索）
```

优点： 遍历不需要栈或递归，直接按线索走即可。

十三、哈夫曼树与哈夫曼编码

13.1 基本概念

- **路径长度：** 从根到结点的分支数
- **带权路径长度（WPL）：** 所有叶子结点的（权值 × 路径长度）之和
- **哈夫曼树：** WPL 最小的二叉树，又称**最优二叉树**

13.2 构造方法

```
步骤：
1. 将 n 个权值作为 n 棵只有根结点的树
2. 选两个最小的树合并（新根 = 两者之和）
3. 重复步骤 2，直到只剩一棵树
```

示例： 权值 {5, 2, 9, 6}

```
第1步：5  2  9  6
        ↘↙
         7
第2步：7  9  6
          ↘↙
          13
第3步：7  9  6 → 合并 7 和 6 → 13
        ↘↙       合并 13 和 9 → 22
        13   9
          ↘↙
          22

最终哈夫曼树（其中一种）：
           22
          /  \
        13    9
       /  \
      7    6
     / \
    5   2

WPL = 5×3 + 2×3 + 6×2 + 9×1 = 15+6+12+9 = 42
```

13.3 哈夫曼编码

- 左分支标 0，右分支标 1（或反过来）
- 从根到叶子的路径上的 0/1 序列就是该字符的编码
- **任意字符的编码不是另一个字符编码的前缀**（前缀编码）

```
上例编码：
5: 000    2: 001    6: 01    9: 1

总编码长度 = WPL = 42 位
```

十四、词频统计问题对比

问题： 统计文件中每个单词的出现次数，按字典序输出。

三种方案对比

方案	查找	插入	空间	实际性能
有序数组	$O(\log_2 n)$ 折半	O(n) 移动	固定大小	插入量大时极慢
链表	O(n) 遍历	O(1)	动态	查找量大时极慢
BST	$O(\log_2 n)$	$O(\log_2 n)$	动态	**综合最优**

> 这也说明了为什么需要学习树结构——线性结构无法同时兼顾"查找快"和"插入快"。

十五、常考题型与易错点

题型 1：性质计算 ⭐⭐⭐⭐⭐

题目： 一棵完全二叉树有 500 个结点，求叶子结点数。

解：

- $h = \lfloor \log_2 500 \rfloor + 1 = \lfloor 8.96 \rfloor + 1 = 9$
- 前 8 层结点数 = $2^8 - 1 = 255$
- 第 9 层结点数 = 500 - 255 = 245
- 第 8 层结点数 = $2^7 = 128$
- 第 8 层有孩子的结点数 = $\lceil 245/2 \rceil = 123$
- 第 8 层叶子数 = 128 - 123 = 5
- 总叶子数 = 5 + 245 = **250**

快速公式： 完全二叉树叶子结点数 = $\lceil n/2 \rceil$。验证：$\lceil 500/2 \rceil = 250$ ✓

题型 2：遍历序列恢复二叉树 ⭐⭐⭐⭐⭐

题目： 前序 ABCDEF，中序 CBAEDF，求后序。

解：

```
前序：A B C D E F → A 是根
中序：C B A E D F → 左子树：C B，右子树：E D F

左子树：前序 B C → B 是根
        中序 C B → C 是 B 的左孩子

右子树：前序 D E F → D 是根
        中序 E D F → E 是 D 的左孩子，F 是 D 的右孩子

二叉树：
      A
     / \
    B   D
   /   / \
  C   E   F

后序：C B E F D A
```

题型 3：BST 的删除 ⭐⭐⭐

题目： 画出删除 50 后的 BST。

```
原始 BST（左<根≤右）：
          50
        /    \
      38      64
     /  \    /  \
   30   40 58   73
```

题型 4：判断 BST/Avl ⭐⭐⭐

题目： 中序遍历一棵树得到递增序列，这棵树是 BST 吗？

答案： **不一定**。中序遍历递增是 BST 的**必要不充分条件**。还需要树的左右子树本身也满足 BST 定义。但通常考研中，如果树的结构本身就是按照 BST 构建的，中序递增就是 BST。

易错点总结

易错点	说明
度为 2 的树 ≠ 二叉树	二叉树严格分左右，树不分
$n_0 = n_2 + 1$ 只适用于二叉树	一般的树没有这个性质
完全二叉树编号从 0 还是 1	从 1 开始：左孩子 2i；从 0 开始：左孩子 2i+1
BST 左<根≤右 还是 左≤根<右	不同定义，注意题目的约定
前序+后序不能唯一确定二叉树	必须要有中序

易错点	说明
递归遍历的返回值	拷贝用前序，删除用后序（想想为什么？）

核心口诀

```
树的结构递归定，前中后序三种遍历。
前序定根中分左右，两者结合唯一确定。
BST 中序递增序，查找插入 logn。
二叉树性质有六个，n0=n2+1 最重要。
完全二叉树编号，左子 2i 右 2i+1。
AVL 平衡因子 ±1，四种旋转要记清。
哈夫曼树求 WPL，最小权值先合并。
```